

Reviews & Ratings APIs

Complete documentation for the review and rating system for events and vendors.

★ Overview

The Reviews & Ratings system provides:

- **Public Review Access:** Browse reviews for events and vendors without authentication
- **Customer Reviews:** Authenticated customers can create, edit, and manage reviews
- **Vendor Responses:** Vendors can respond to reviews about their events
- **Review Moderation:** Admin tools for review approval and content moderation
- **Rating Analytics:** Comprehensive rating statistics and insights

📖 Public Review Access

GET /api/reviews/:type/:id

Get reviews for a specific event or vendor with filtering options.

Authentication: Not required

Path Parameters:

- **type** (string): Review target type ("event" or "vendor")
- **id** (string): MongoDB ObjectId of the target (event ID or vendor ID)

Query Parameters:

- **rating** (number): Filter by specific rating (1-5)
- **verified** (boolean): Show only verified reviews ("true"/"false")
- **sortBy** (string): Sort field ("createdAt", "rating", "helpfulness", default: "createdAt")
- **sortOrder** (string): Sort order ("asc", "desc", default: "desc")
- **page** (number): Page number (default: 1)
- **limit** (number): Items per page (default: 10, max: 50)
- **withMedia** (boolean): Include only reviews with photos/videos ("true"/"false")

Example Request:

```
curl "http://localhost:5000/api/reviews/event/68a155c40214f9e8ddf64aff?rating=5&verified=true&limit=5"
```

Response:

```
{
  "success": true,
  "message": "Reviews retrieved successfully",
  "data": {
    "reviews": [
      {
        "_id": "review_id_123",
        "type": "event",
        "targetId": "68a155c40214f9e8ddf64aff",
        "customerId": {
          "_id": "customer_id",
          "firstName": "Sarah",
          "lastName": "Ahmed",
          "avatar": "https://cloudinary.com/customer-avatar.jpg",
          "isVerified": true
        },
        "rating": 5,
        "title": "Absolutely Amazing Experience!",
        "comment": "This was hands down the best music festival I've ever attended. The organization was flawless, the sound quality was incredible, and the venue was perfect.",
        "pros": [
          "Excellent sound quality",
          "Great organization",
          "Beautiful venue"
        ]
      }
    ]
  }
}
```

```

        "Diverse lineup"
    ],
    "cons": [
        "Long food lines",
        "Parking was limited"
    ],
    "isVerified": true,
    "orderId": "order_id_456",
    "media": [
        {
            "type": "image",
            "url": "https://cloudinary.com/review-image-1.jpg",
            "caption": "Main stage view"
        },
        {
            "type": "image",
            "url": "https://cloudinary.com/review-image-2.jpg",
            "caption": "Amazing crowd"
        }
    ],
    "helpfulVotes": {
        "helpful": 24,
        "notHelpful": 2,
        "total": 26
    },
    "vendorResponse": {
        "message": "Thank you so much for your wonderful review! We're thrilled you had such a great
time.",
        "respondedAt": "2024-03-17T10:30:00.000Z",
        "respondedBy": {
            "firstName": "Ahmed",
            "lastName": "Events"
        }
    },
    "status": "approved",
    "createdAt": "2024-03-16T15:30:00.000Z",
    "updatedAt": "2024-03-16T15:30:00.000Z"
}
},
"summary": {
    "totalReviews": 156,
    "averageRating": 4.6,
    "ratingDistribution": {
        "5": 89,
        "4": 45,
        "3": 15,
        "2": 5,
        "1": 2
    },
    "verifiedPercentage": 78.5
},
"pagination": {
    "currentPage": 1,
    "totalPages": 32,
    "totalReviews": 156,
    "hasNextPage": true,
    "hasPrevPage": false,
    "limit": 5
}
}
}

```

Customer Review Management

POST /api/reviews

Create a new review for an event or vendor.

Authentication: Required (Customer role)

Headers:

Authorization: Bearer <access_token>
Content-Type: application/json

Request Body:

```
{
  "type": "event",
  "targetId": "68a155c40214f9e8ddf64aff",
  "rating": 5,
  "title": "Fantastic Event Experience",
  "comment": "Had an absolutely wonderful time at this event. Everything was well organized and the entertainment was top-notch.",
  "pros": [
    "Excellent organization",
    "Great entertainment",
    "Good value for money",
    "Friendly staff"
  ],
  "cons": [
    "Long entry queues",
    "Limited parking"
  ],
  "orderId": "order_id_optional",
  "media": [
    "https://cloudinary.com/user-review-photo-1.jpg",
    "https://cloudinary.com/user-review-photo-2.jpg"
  ],
  "wouldRecommend": true,
  "attendedDate": "2024-03-15T18:00:00.000Z"
}
```

Response:

```
{
  "success": true,
  "message": "Review submitted successfully and is pending moderation",
  "data": {
    "review": {
      "_id": "new_review_id",
      "type": "event",
      "targetId": "68a155c40214f9e8ddf64aff",
      "customerId": "customer_id",
      "rating": 5,
      "title": "Fantastic Event Experience",
      "comment": "Had an absolutely wonderful time...",
      "pros": ["Excellent organization", "Great entertainment"],
      "cons": ["Long entry queues", "Limited parking"],
      "isVerified": true,
      "status": "pending",
      "createdAt": "2024-03-16T16:00:00.000Z"
    }
  }
}
```

GET /api/reviews/my-reviews

Get all reviews created by the authenticated user.

Authentication: Required (Customer role)

Headers:

Authorization: Bearer <access_token>

Query Parameters:

- type (string): Filter by review type ("event", "vendor")

- **status** (string): Filter by status ("pending", "approved", "rejected")
- **rating** (number): Filter by rating (1-5)
- **page** (number): Page number (default: 1)
- **limit** (number): Items per page (default: 10)
- **sortBy** (string): Sort field ("createdAt", "rating", "title")
- **sortOrder** (string): Sort order ("asc", "desc", default: "desc")

Response:

```
{
  "success": true,
  "message": "User reviews retrieved successfully",
  "data": {
    "reviews": [
      {
        "_id": "review_id",
        "type": "event",
        "targetId": {
          "_id": "event_id",
          "title": "Summer Music Festival",
          "images": ["https://cloudinary.com/event-image.jpg"]
        },
        "rating": 5,
        "title": "Amazing Experience",
        "comment": "Loved every moment...",
        "status": "approved",
        "helpfulVotes": {
          "helpful": 12,
          "notHelpful": 1
        },
        "vendorResponse": {
          "message": "Thank you for the wonderful review!",
          "respondedAt": "2024-03-17T10:30:00.000Z"
        },
        "createdAt": "2024-03-16T15:30:00.000Z"
      }
    ],
    "summary": {
      "totalReviews": 8,
      "averageRating": 4.4,
      "pendingReviews": 1,
      "approvedReviews": 6,
      "rejectedReviews": 1
    },
    "pagination": {...}
  }
}
```

PUT /api/reviews/:id

Update an existing review (within 24 hours of creation).

Authentication: Required (Review author only)

Headers:

Authorization: Bearer <access_token> Content-Type: application/json
--

Path Parameters:

- **id** (string): Review MongoDB ObjectId

Request Body: Same structure as POST /api/reviews (partial updates allowed)

Response:

```
{
  "success": true,
  "message": "Review updated successfully",
  "data": {
    "review": {
      "_id": "review_id",
      "rating": 4,
      "title": "Updated Review Title",
      "comment": "Updated review content...",
      "status": "pending",
      "updatedAt": "2024-03-16T17:00:00.000Z"
    }
  }
}
```

Note: Reviews can only be edited within 24 hours of creation. Major edits may require re-moderation.

DELETE /api/reviews/:id

Delete a review (author or admin only).

Authentication: Required (Review author or Admin)

Headers:

```
Authorization: Bearer <access_token>
```

Path Parameters:

- `id` (string): Review MongoDB ObjectId

Response:

```
{
  "success": true,
  "message": "Review deleted successfully"
}
```

Review Interaction System

POST /api/reviews/:id/vote

Vote on review helpfulness.

Authentication: Required (Any authenticated user)

Headers:

```
Authorization: Bearer <access_token>
Content-Type: application/json
```

Path Parameters:

- `id` (string): Review MongoDB ObjectId

Request Body:

```
{
  "helpful": true
}
```

Note: Use `true` for helpful, `false` for not helpful. Users can change their vote.

Response:

```
{
  "success": true,
  "message": "Vote recorded successfully",
  "data": {
    "review": {
      "_id": "review_id",
      "helpfulVotes": {
        "helpful": 25,
        "notHelpful": 2,
        "total": 27
      }
    },
    "userVote": "helpful"
  }
}
```

POST /api/reviews/:id/flag

Flag a review for moderation.

Authentication: Required (Any authenticated user)

Headers:

```
Authorization: Bearer <access_token>
Content-Type: application/json
```

Path Parameters:

- id (string): Review MongoDB ObjectId

Request Body:

```
{
  "reason": "spam",
  "description": "This review appears to be spam content with promotional links.",
  "evidence": [
    "https://cloudinary.com/evidence-screenshot.jpg"
  ]
}
```

Available reasons: "inappropriate", "spam", "fake", "offensive", "copyright", "personal_info", "other"

Response:

```
{
  "success": true,
  "message": "Review flagged successfully. Our moderation team will review it shortly.",
  "data": {
    "flag": {
      "_id": "flag_id",
      "reviewId": "review_id",
      "reason": "spam",
      "status": "pending",
      "submittedAt": "2024-03-16T18:00:00.000Z"
    }
  }
}
```

Vendor Response System

POST /api/reviews/:id/respond

Respond to a review as a vendor.

Authentication: Required (Vendor role - event/venue owner only)

Headers:

```
Authorization: Bearer <access_token>
Content-Type: application/json
```

Path Parameters:

- `id` (string): Review MongoDB ObjectId

Request Body:

```
{
  "message": "Thank you so much for your wonderful review! We're delighted to hear you had such a great experience. Your feedback about the parking situation is valuable and we're working on expanding our parking facilities for future events.",
  "isPublic": true,
  "followUpAction": "Contacting customer privately about parking refund"
}
```

Response:

```
{
  "success": true,
  "message": "Response posted successfully",
  "data": {
    "response": {
      "_id": "response_id",
      "reviewId": "review_id",
      "vendorId": "vendor_id",
      "message": "Thank you so much for your wonderful review...",
      "isPublic": true,
      "respondedAt": "2024-03-16T18:30:00.000Z"
    }
  }
}
```

Notes:

- Only the vendor who owns the event/venue can respond
- Responses are publicly visible
- Vendors should maintain professional tone in responses

Admin Review Moderation

GET /api/reviews/admin/pending

Get reviews pending moderation.

Authentication: Required (Admin role)

Headers:

```
Authorization: Bearer <access_token>
```

Query Parameters:

- `type` (string): Filter by review type ("event", "vendor")
- `priority` (string): Filter by priority ("high", "medium", "low")
- `flagged` (boolean): Show only flagged reviews ("true"/"false")
- `page` (number): Page number (default: 1)
- `limit` (number): Items per page (default: 20, max: 100)
- `sortBy` (string): Sort field ("createdAt", "flagCount", "rating")

Response:

```
{
  "success": true,
  "message": "Pending reviews retrieved successfully",
  "data": {
    "reviews": [
      {
        "_id": "review_id",
        "type": "event",
        "targetId": {
          "title": "Event Title",
          "vendorId": {
            "firstName": "Vendor",
            "lastName": "Name"
          }
        },
      },
      "customerId": {
        "firstName": "Customer",
        "lastName": "Name",
        "email": "customer@example.com"
      },
      "rating": 1,
      "title": "Terrible Experience",
      "comment": "Review content here...",
      "flags": [
        {
          "reason": "inappropriate",
          "description": "Contains inappropriate language",
          "submittedBy": "user_id",
          "submittedAt": "2024-03-16T17:00:00.000Z"
        }
      ],
      "priority": "high",
      "autoModeration": {
        "toxicityScore": 0.85,
        "recommendations": ["manual_review_required"]
      },
      "createdAt": "2024-03-16T16:30:00.000Z"
    ],
    "summary": {
      "totalPending": 23,
      "highPriority": 5,
      "flaggedReviews": 8,
      "autoFlagged": 3
    },
    "pagination": {...}
  }
}
```

PUT /api/reviews/admin/:id/moderate

Moderate a review (approve, reject, or request changes).

Authentication: Required (Admin role)

Headers:

Authorization: Bearer <access_token> Content-Type: application/json
--

Path Parameters:

- `id` (string): Review MongoDB ObjectId

Request Body:


```
{
  "status": "approved",
  "notes": "Review approved after verification. Customer provided valid order proof.",
  "action": "approve",
  "moderatorComment": "Legitimate review from verified customer",
  "notifyCustomer": true,
  "notifyVendor": true
}
```

Available statuses: "pending", "approved", "rejected", "requires_changes"

Response:

```
{
  "success": true,
  "message": "Review moderation completed successfully",
  "data": {
    "review": {
      "_id": "review_id",
      "status": "approved",
      "moderatedBy": "admin_id",
      "moderatedAt": "2024-03-16T19:00:00.000Z",
      "moderationNotes": "Review approved after verification..."
    },
    "notifications": {
      "customerNotified": true,
      "vendorNotified": true
    }
  }
}
```

Review Analytics

GET /api/reviews/analytics/:type/:id

Get comprehensive review analytics for events or vendors.

Authentication: Required (Vendor for own events/Admin for all)

Headers:

```
Authorization: Bearer <access_token>
```

Path Parameters:

- `type` (string): Analytics target ("event", "vendor")
- `id` (string): Target MongoDB ObjectId

Query Parameters:

- `period` (number): Analysis period in days (default: 30, max: 365)
- `detailed` (boolean): Include detailed breakdown ("true"/"false")

Response:

```

{
  "success": true,
  "message": "Review analytics retrieved successfully",
  "data": {
    "overview": {
      "totalReviews": 156,
      "averageRating": 4.6,
      "responseRate": 87.2,
      "verifiedPercentage": 78.5
    },
    "ratingDistribution": {
      "5": { "count": 89, "percentage": 57.1 },
      "4": { "count": 45, "percentage": 28.8 },
      "3": { "count": 15, "percentage": 9.6 },
      "2": { "count": 5, "percentage": 3.2 },
      "1": { "count": 2, "percentage": 1.3 }
    },
    "trends": {
      "ratingTrend": 0.15,
      "reviewVolumeTrend": 0.08,
      "period": "30_days"
    },
    "sentimentAnalysis": {
      "positive": 142,
      "neutral": 11,
      "negative": 3,
      "positivePercentage": 91.0
    },
    "commonThemes": {
      "positive": [
        { "theme": "organization", "mentions": 67 },
        { "theme": "entertainment", "mentions": 54 },
        { "theme": "value", "mentions": 43 }
      ],
      "negative": [
        { "theme": "parking", "mentions": 12 },
        { "theme": "queues", "mentions": 8 }
      ]
    },
    "timelineData": [
      {
        "date": "2024-03-01",
        "reviews": 8,
        "averageRating": 4.4
      }
    ]
  }
}

```

Implementation Examples

Review Display Component

```

// React component for displaying reviews
const ReviewCard = ({ review, onVote, onFlag, onRespond }) => {
  const [userVote, setUserVote] = useState(null);

  const handleVote = async (helpful) => {
    try {
      await onVote(review._id, helpful);
      setUserVote(helpful ? 'helpful' : 'not_helpful');
    } catch (error) {
      console.error('Vote failed:', error);
    }
  };

  return (

```

```

<div className="review-card">
  <div className="review-header">
    <div className="customer-info">
      <img src={review.customerId.avatar} alt="Customer" />
      <div>
        <h4>{review.customerId.firstName} {review.customerId.lastName}</h4>
        {review.isVerified && <span className="verified">✓ Verified Purchase</span>}
      </div>
    </div>
    <div className="rating">
      {'★'.repeat(review.rating)}{'★'.repeat(5 - review.rating)}
    </div>
  </div>

  <h3>{review.title}</h3>
  <p>{review.comment}</p>

  {review.pros.length > 0 && (
    <div className="pros">
      <h5>👍 Pros:</h5>
      <ul>{review.pros.map(pro => <li key={pro}>{pro}</li>)}</ul>
    </div>
  )}

  {review.cons.length > 0 && (
    <div className="cons">
      <h5>👎 Cons:</h5>
      <ul>{review.cons.map(con => <li key={con}>{con}</li>)}</ul>
    </div>
  )}

  {review.media && review.media.length > 0 && (
    <div className="review-media">
      {review.media.map((media, index) => (
        <img key={index} src={media.url} alt={media.caption} />
      ))}
    </div>
  )}

  {review.vendorResponse && (
    <div className="vendor-response">
      <h5>Response from {review.vendorResponse.respondedBy.firstName}</h5>
      <p>{review.vendorResponse.message}</p>
    </div>
  )}

  <div className="review-actions">
    <button
      onClick={() => handleVote(true)}
      className={userVote === 'helpful' ? 'active' : ''}
    >
      👍 Helpful ({review.helpfulVotes.helpful})
    </button>
    <button
      onClick={() => handleVote(false)}
      className={userVote === 'not_helpful' ? 'active' : ''}
    >
      👎 Not Helpful ({review.helpfulVotes.notHelpful})
    </button>
    <button onClick={() => onFlag(review._id)}>
      🚩 Flag
    </button>
  </div>
</div>
);
};

```

Review Form Component

```
// Review creation form
const ReviewForm = ({ eventId, onSubmit }) => {
  const [formData, setFormData] = useState({
    rating: 0,
    title: '',
    comment: '',
    pros: [],
    cons: [],
    wouldRecommend: true
  });

  const handleSubmit = async (e) => {
    e.preventDefault();

    try {
      const response = await fetch('/api/reviews', {
        method: 'POST',
        headers: {
          'Authorization': `Bearer ${accessToken}`,
          'Content-Type': 'application/json'
        },
        body: JSON.stringify({
          type: 'event',
          targetId: eventId,
          ...formData
        })
      });

      const result = await response.json();

      if (result.success) {
        onSubmit(result.data.review);
        setFormData({ rating: 0, title: '', comment: '', pros: [], cons: [], wouldRecommend: true });
      }
    } catch (error) {
      console.error('Review submission failed:', error);
    }
  };

  return (
    <form onSubmit={handleSubmit} className="review-form">
      <div className="rating-input">
        <label>Overall Rating:</label>
        <div className="star-rating">
          {[1, 2, 3, 4, 5].map(star => (
            <button
              key={star}
              type="button"
              onClick={() => setFormData(prev => ({ ...prev, rating: star })}
              className={star <= formData.rating ? 'active' : ''}
            >
              ★
            </button>
          ))}
        </div>
      </div>

      <input
        type="text"
        placeholder="Review title"
        value={formData.title}
        onChange={(e) => setFormData(prev => ({ ...prev, title: e.target.value })}
        required
      />

      <textarea
        placeholder="Share your experience..."
        value={formData.comment}
        onChange={(e) => setFormData(prev => ({ ...prev, comment: e.target.value })}

```

```
        required
      />

      <div className="recommendation">
        <label>
          <input
            type="checkbox"
            checked={formData.wouldRecommend}
            onChange={ (e) => setFormData(prev => ({ ...prev, wouldRecommend: e.target.checked })))}
          />
          I would recommend this event
        </label>
      </div>

      <button type="submit" disabled={!formData.rating || !formData.title || !formData.comment}>
        Submit Review
      </button>
    </form>
  );
};
```

Related Documentation:

- [Events → \(events.md\)](#) - Event information and management
- [Venues & Vendors → \(venues-vendors.md\)](#) - Vendor profile and venue information
- [Orders & Payments → \(orders-payments.md\)](#) - Order verification for review authenticity
- [Admin APIs → \(admin-apis.md\)](#) - Administrative review management functions